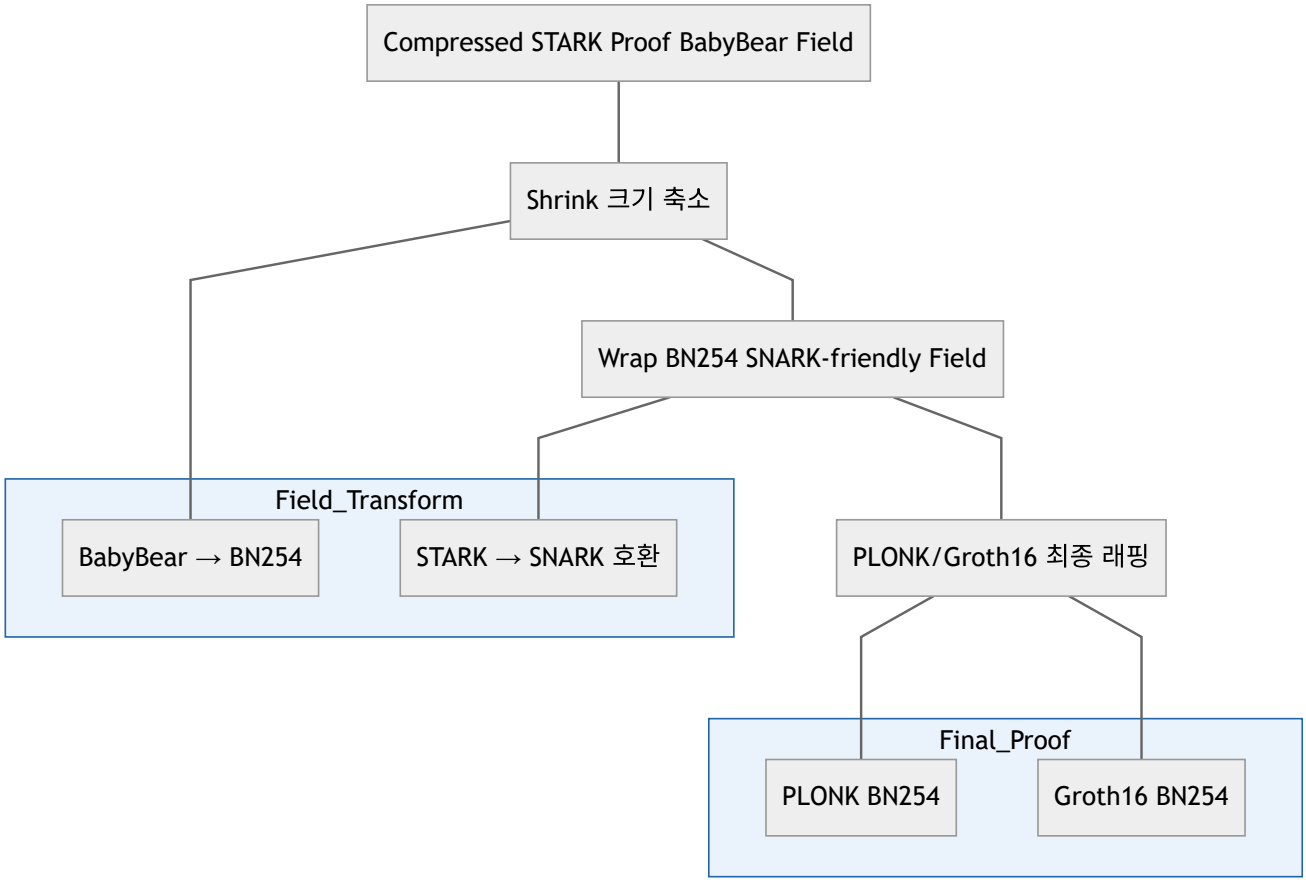
08. SNARK 래핑 과정 분석 (SNARK Wrapping Process Analysis)

개요

SP1의 SNARK 래핑 과정은 STARK 증명을 SNARK-friendly 필드로 변환하고, 최종적으로 Groth16 또는 PLONK 증명으로 래핑하는 과정입니다. 이 문서는 STARK에서 Groth16/PLONK으로의 변환 과정, BN254 곡선 기반 증명 생성, 그리고 최종 증명 검증 과정을 상세히 분석합니다.

1. SNARK 래핑 전체 아키텍처

1.1 래핑 파이프라인 구조



1.2 래핑 단계별 구성

SP1의 SNARK 래핑은 다음과 같은 3단계로 구성됩니다:

1. **Shrink**: 압축된 증명의 크기를 더욱 축소
2. **Wrap BN254**: SNARK-friendly BN254 필드로 변환
3. **Final Wrap**: Groth16 또는 PLONK 증명으로 최종 래핑

2. Shrink 과정 상세 분석

2.1 shrink 함수 구조

```
pub fn shrink(
&self,
reduced_proof: SP1ReduceProof<InnerSC>,
opts: SP1ProverOpts,
) -> Result<SP1ReduceProof<InnerSC>, SP1RecursionProverError>
```

주요 단계:

1. **입력 준비**

```
let SP1ReduceProof { vk: compressed_vk, proof: compressed_proof } = reduced_proof;
let input = SP1CompressWitnessValues {
vks_and_proofs: vec![(compressed_vk.clone(), compressed_proof)],
is_complete: true,
};
```

2. **머클 증명 생성**

```
let input_with_merkle = self.make_merkle_proofs(input);
```

3. **Shrink 프로그램 실행**

```
let program = self.shrink_program(ShrinkAir::<BabyBear>::shrink_shape(), &input_with_merkl
```

2.2 Shrink 프로그램 실행

재귀 런타임 설정:

```
let mut runtime = RecursionRuntime::<Val<InnerSC>, Challenge<InnerSC>, _>::new(
program.clone(),
self.shrink_prover.config().perm.clone(),
);
```

위트니스 스트림 처리:

```
let mut witness_stream = Vec::new();
Witnessable::<InnerConfig>::write(&input_with_merkle, &mut witness_stream);
runtime.witness_stream = witness_stream.into();
runtime.run().unwrap();
```

증명 생성:

```
let (shrink_pk, shrink_vk) = self.shrink_prover.setup(&program);
let mut compress_challenger = self.shrink_prover.config().challenger();
let mut compress_proof = self
.shrink_prover
.prove(&shrink_pk, vec![runtime.record], &mut compress_challenger, opts.recursion_opts)
.unwrap();
```

3. BN254 필드로의 변환 (Wrap BN254)

3.1 wrap_bn254 함수 분석

```
pub fn wrap_bn254(
&self,
compressed_proof: SP1ReduceProof<InnerSC>,
opts: SP1ProverOpts,
) -> Result<SP1ReduceProof<OuterSC>, SP1RecursionProverError>
```

핵심 특징:

- **필드 변환**: `InnerSC` (BabyBear) → `OuterSC` (BN254)
- **SNARK 호환성**: BN254 곡선 기반 증명 생성
- **크기 최적화**: 최종 래핑을 위한 증명 크기 최적화

3.2 BN254 필드 구성

OuterSC 타입 정의:

```
pub type OuterSC = BabyBearPoseidon2Outer;
```

BabyBearPoseidon2Outer 구조:

```
pub struct BabyBearPoseidon2Outer {
pub perm: OuterPerm,
pub pcs: OuterPcs,
}

impl StarkGenericConfig for BabyBearPoseidon2Outer {
type Val = OuterVal; // BN254 스칼라 필드
type Domain = <OuterPcs as Pcs<OuterChallenge, OuterChallenger>>::Domain;
```

```
type Pcs = OuterPcs;

type Challenge = OuterChallenge;

type Challenger = OuterChallenger;

}
```

3.3 필드 변환 과정

1단계: 입력 준비

```
let SP1ReduceProof { vk: compressed_vk, proof: compressed_proof } = compressed_proof;

let  input = SP1CompressWitnessValues {

vks_and_proofs: vec![(compressed_vk, compressed_proof)],

is_complete: true,

};
```

2단계: Wrap 프로그램 실행

```
let  program = self.wrap_program();

let  mut  runtime = RecursionRuntime::<Val<InnerSC>, Challenge<InnerSC>, _>::new(

program.clone(),

self.shrink_prover.config().perm.clone(),

);
```

3단계: BN254 필드에서 증명 생성

```
let (wrap_pk, wrap_vk) = self.wrap_prover.setup(&program);

let  mut  wrap_challenger = self.wrap_prover.config().challenger();

let  mut  wrap_proof = self

.wrap_prover
```

```
.prove(&wrap_pk, vec![runtime.record], &mut wrap_challenger, opts.recursion_opts)
.unwrap();
```

4. PLONK 증명 생성

4.1 wrap_plonk_bn254 함수 분석

```
pub fn wrap_plonk_bn254(
&self,
proof: SP1ReduceProof<OuterSC>,
build_dir: &Path,
) -> PlonkBn254Proof
```

주요 구성 요소:

1. **공개 입력 생성**

```
let vkey_hash = sp1_vkey_digest_bn254(&proof);
let committed_values_digest = sp1_committed_values_digest_bn254(&proof);
```

2. **위트니스 준비**

```
let mut witness = Witness::default();
input.write(&mut witness);
witness.write_committed_values_digest(committed_values_digest);
witness.write_vkey_hash(vkey_hash);
```

3. **PLONK 증명 생성**

```
let prover = PlonkBn254Prover::new();

let proof = prover.prove(witness, build_dir.to_path_buf());
```

4.2 PlonkBn254Prover 구조

프로버 정의:

```
pub struct PlonkBn254Prover;


impl PlonkBn254Prover {

pub fn new() -> Self {

Self

}

pub fn prove(&self, witness: Witness, build_dir: PathBuf) -> PlonkBn254Proof {

// Gnark FFI를 통한 PLONK 증명 생성

}

pub fn verify(

&self,

proof: &PlonkBn254Proof,

vkey_hash: &BigUint,

committed_values_digest: &BigUint,

build_dir: &Path,

) -> Result<()> {

// PLONK 증명 검증

}

}
```

4.3 PLONK 증명 구조

증명 데이터:

```
pub struct PlonkBn254Proof {
pub public_inputs: Vec<String>,
pub encoded_proof: String,
pub raw_proof: String,
pub plonk_vkey_hash: [u8; 32],
}
```

공개 입력:

- `vkey_hash` : SP1 검증 키의 해시
- `committed_values_digest` : 커밋된 공개 값들의 다이제스트

5. Groth16 증명 생성

5.1 wrap_groth16_bn254 함수 분석

```
pub fn wrap_groth16_bn254(
&self,
proof: SP1ReduceProof<OuterSC>,
build_dir: &Path,
) -> Groth16Bn254Proof
```

PLONK와의 차이점:

- **증명 크기**: Groth16이 더 작은 증명 크기
- **검증 시간**: Groth16이 더 빠른 검증
- **신뢰 설정**: Groth16은 회로별 신뢰 설정 필요

5.2 Groth16Bn254Prover 구조

```
pub struct Groth16Bn254Prover;

impl Groth16Bn254Prover {
pub fn prove(&self, witness: Witness, build_dir: PathBuf) -> Groth16Bn254Proof {
// Gnark FFI를 통한 Groth16 증명 생성
}
pub fn verify(
&self,
proof: &Groth16Bn254Proof,
vkey_hash: &BigUint,
committed_values_digest: &BigUint,
build_dir: &Path,
) -> Result<()> {
// Groth16 증명 검증
}
}
```

5.3 Groth16 증명 구조

```
pub struct Groth16Bn254Proof {
pub public_inputs: Vec<String>,
pub encoded_proof: String,
pub raw_proof: String,
}
```

6. BN254 곡선 기반 암호학

6.1 BN254 곡선 특성

곡선 방정식:

- $y^2 = x^3 + 3$ (over $\mathbb{F}_p$)
- $p = 21888242871839275222246405745257275088696311157297823662689037894645226208583$

페어링 친화적 특성:

- Type-3 페어링 지원
- 128비트 보안 수준
- SNARK 시스템에 최적화

6.2 필드 연산 최적화

스칼라 필드 연산:

```
pub type OuterVal = Bn254Fr; // BN254 스칼라 필드
```

해시 함수 최적화:

```
impl FieldHasher<BabyBear> for BabyBearPoseidon2Outer {
type Digest = [Bn254Fr; BN254_DIGEST_SIZE];
fn hash(input: &[BabyBear]) -> Self::Digest {
// BabyBear → BN254 필드 해시
}
}
```

6.3 페어링 연산

페어링 함수:

- $e : G_1 \times G_2 \to G_T$
- 이중선형성: $e(aP, bQ) = e(P, Q)^{ab}$
- 비퇴화성: $e(P, Q) \neq 1$ for non-trivial P, Q

7. Gnark FFI 통합

7.1 FFI 인터페이스

Gnark 바인딩:

```
// Gnark Go 코드와의 FFI 인터페이스
extern "C" {
fn prove_plonk_bn254(
witness_ptr: *const u8,
witness_len: usize,
build_dir_ptr: *const c_char,
) -> *mut PlonkProofResult;
fn prove_groth16_bn254(
witness_ptr: *const u8,
witness_len: usize,
build_dir_ptr: *const c_char,
) -> *mut Groth16ProofResult;
}
```

7.2 위트니스 직렬화

위트니스 포맷:

```
pub struct Witness {
```

```
pub vars: Vec<Bn254Fr>,
pub felts: Vec<Vec<Bn254Fr>>,
pub exts: Vec<Vec<Bn254Fr>>,
pub commits: Vec<Vec<Bn254Fr>>,
}

impl Witness {
pub fn write_committed_values_digest(&mut self, digest: Bn254Fr) {
self.commits.push(vec![digest]);
}
pub fn write_vkey_hash(&mut self, hash: Bn254Fr) {
self.vars.push(hash);
}
}
```

7.3 증명 검증

온체인 검증:

```
contract SP1Verifier {
function verifyProof(
bytes32 vkeyHash,
bytes32 publicValuesDigest,
bytes calldata proofBytes
) external view returns (bool) {
// Groth16/PLONK 증명 검증
}
```

```
}
```

8. 성능 최적화 전략

8.1 회로 최적화

제약 조건 최소화:

- 불필요한 제약 조건 제거
- 효율적인 게이트 사용
- 병렬 처리 가능한 구조 설계

메모리 최적화:

```
// 지연 삭제를 통한 메모리 관리
rayon::spawn(move || {
drop(runtime.record);
});
```

8.2 빌드 시간 최적화

사전 컴파일:

```
// 회로 사전 빌드
PlonkBn254Prover::build(constraints, witness, build_dir);
Groth16Bn254Prover::build(constraints, witness, build_dir);
```

캐싱 전략:

- 컴파일된 회로 캐싱
- 신뢰 설정 재사용
- 증명 키 사전 생성

8.3 병렬 처리

멀티스레드 증명 생성:

```
// 병렬 위트니스 생성
witness_parts.par_iter().map(|part| {
generate_witness_part(part)
}).collect()
```

9. 보안 고려사항

9.1 신뢰 설정

Groth16 신뢰 설정:

- 회로별 신뢰 설정 필요
- Powers of Tau 의식 참여
- 독성 폐기물(toxic waste) 제거

PLONK 범용 설정:

- 회로 독립적 신뢰 설정
- 한 번의 설정으로 모든 회로 지원
- 더 높은 유연성

9.2 암호학적 가정

이산 로그 문제:

- BN254 곡선에서의 ECDLP
- 128비트 보안 수준
- 양자 컴퓨터 위협 고려

페어링 가정:

- 이중선형 Diffie-Hellman 가정
- 지식 추출 가정
- 무작위 오라클 모델

10. 검증 과정 분석

10.1 증명 검증 단계

1단계: 공개 입력 검증

```
let vkey_hash = BigUint::from_str(&proof.public_inputs[0])?;
let committed_values_digest = BigUint::from_str(&proof.public_inputs[1])?;
```

2단계: 증명 구조 검증

```
prover.verify(
&proof,
&vkey_hash.as_canonical_biguint(),
&committed_values_digest.as_canonical_biguint(),
build_dir,
)?;
```

3단계: 페어링 검증

- Groth16: 3개의 페어링 연산
- PLONK: 다수의 페어링 연산 (배치 최적화 가능)

10.2 온체인 검증

가스 비용 최적화:

- 사전 컴파일된 페어링 함수 사용

- 배치 검증을 통한 비용 절약
- 효율적인 스칼라 곱셈 구현

검증 시간:

- Groth16: ~1-2ms (오프체인), ~300k gas (온체인)
- PLONK: ~5-10ms (오프체인), ~500k gas (온체인)

11. 결론

SP1의 SNARK 래핑 과정은 다음과 같은 핵심 특징을 가집니다:

1. **다단계 변환**: STARK → Shrink → Wrap → SNARK
2. **필드 호환성**: BabyBear → BN254 필드 변환
3. **유연한 백엔드**: PLONK와 Groth16 모두 지원
4. **최적화**: 회로 크기 및 증명 시간 최적화
5. **온체인 호환**: 이더리움 등 블록체인에서 검증 가능

이러한 설계를 통해 SP1은 STARK의 투명성과 확장성을 유지하면서도, SNARK의 간결성과 빠른 검증 시간을 동시에 제공할 수 있습니다. 특히 BN254 곡선 기반의 구현은 기존 블록체인 인프라와의 호환성을 보장하여 실용적인 영지식 증명 시스템을 구현합니다.